MethodTaskSelectionModal UI設計書

メタデータ

- **コンポーネントID**: MethodTaskSelectionModal
- **コンポーネント名**: メソッド・タスク選択モーダル
- **ファイルパス**: `frontend/src/components/ExperimentWorker/MethodTaskSelectionModal.tsx`
- **利用権限**: 実験作業者（experiment_worker）
- **作成日**: 2026-01-19
- **最終更新日**: 2026-01-19

---

1. 概要

1.1 コンポーネントの目的

メソッドとタスクを一度に選択できるモーダルです。進捗確認タブと同様の構造（メソッドタブ + タスクテーブル）を持ち、メソッドを切り替えながらタスクを選択できます。

1.2 主要機能

- メソッドタブによるメソッド切り替え
- タスクテーブル表示（ラジオボタン選択）
- 検索・フィルタリング機能（全カラム検索、タスク名、ステータス）
- ソート機能（タスク名、ステータス、進捗率）
- ページネーション（10, 20, 50件/ページ）
- 選択解除ボタン
- 一時選択状態管理（決定ボタンで確定）

1.3 使用場所

- ExperimentRecordTab（実験記録タブ）
- ProgressCheckTab（進捗確認タブ）

- InstructionConfirmationTab（指示確認タブ）

1.4 他の選択モーダルとの違い

項目	Project/Method	Task	MethodTask
表示形式	Table	List	Tab + Table
選択対象	単一（Project or Method）	単一（Task）	複数（Method + Task）
選択UI	ラジオボタン	クリック選択	ラジオボタン
フィルタリング	あり	なし	あり
タブ切り替え	なし	なし	あり

---

2. コンポーネント構成

2.1 画面イメージ


メソッド・タスク選択モーダルイメージ

2.2 レイアウト構造

```
┌─Modal──────────────────────────────────────────────┐
│ Header: [CheckOutlined] メソッド・タスク選択          │
├────────────────────────────────────────────────────┤
│ Body: (maxHeight: 70vh、スクロール可能)               │
│ ┌─Tabs──────────────────────────────────────────┐ │
│ │ [メソッド1] [メソッド2] [メソッド3] ...          │ │
│ ├──────────────────────────────────────────────┤ │
│ │ Tab Content (アクティブなメソッドのタスク一覧)    │ │
│ │                                              │ │
│ │ ┌─検索・フィルターUI──────────────────────┐   │ │
│ │ │ [検索...] [タスク名▼] [ステータス▼]      │   │ │
│ │ └──────────────────────────────────────┘   │ │
│ │                                              │ │
│ │ ┌─Table──────────────────────────────────┐   │ │
│ │ │ ○ タスク名    ステータス  進捗率 │        │   │ │
│ │ │ ◉ タスクA     [開始]      50%   │ ← 選択中 │   │ │
│ │ │ ○ タスクB     [完了]      100%  │        │   │ │
│ │ │ ○ タスクC     [作業中]    30%   │        │   │ │
│ │ └──────────────────────────────────────┘   │ │
│ │ Pagination: [1] [2] [3] ... [10] 全30件      │ │
```

```
│ └──────────────────────────────────────────────────┘ │
├──────────────────────────────────────────────────────┤
│ Footer:                                              │
│  [選択解除]          [キャンセル] [決定]             │
│   (danger)             (default)  (primary)          │
└──────────────────────────────────────────────────────┘

モーダル幅: 900px
ボディ最大高さ: 70vh（スクロール可能）
```

2.3 使用コンポーネント

Ant Design コンポーネント

- `Modal` - モーダルダイアログ
- `Tabs` - メソッドタブ切り替え
- `Table` - タスク一覧テーブル
- `Radio` - タスク選択ラジオボタン
- `Tag` - ステータスタグ
- `Input` - 検索入力
- `Select` - タスク名・ステータスフィルター
- `Button` - 選択解除、キャンセル、決定ボタン
- `Row` / `Col` - レスポンシブグリッド
- `Space` - 要素間隔制御
- `Typography.Text` - テキスト表示
- `theme` - デザイントークン取得

アイコン

- `CheckOutlined` - 決定アイコン
- `CloseOutlined` - 選択解除アイコン
- `SearchOutlined` - 検索アイコン

---

3. Props

```
interface MethodTaskSelectionModalProps {
  open: boolean;                                    // モーダルの開閉状態
  onClose: () => void;                              // 閉じるイベントハンドラ
  selectedProject: string;                          // 選択中のプロジェクトID
  selectedMethodId: string | null;                  // 選択中のメソッドID
  selectedTaskId: string | null;                    // 選択中のタスクID
  onSelect: (methodId: string, taskId: string) => void;  // メソッド・タスク選択イベント
  methods: Method[];                                // APIから取得したメソッド一覧
}
```

重要な前提条件:

- `selectedProject` が指定されている前提
- メソッド一覧（`methods`）は呼び出し元（親コンポーネント）からpropsとして渡される
- メソッドIDとタスクIDを同時に返す

---

4. State（内部状態）

4.1 選択状態

```
// 一時選択状態（決定ボタン押下時に確定）
const [tempSelectedMethodId, setTempSelectedMethodId] = useState<string | null>(null);
const [tempSelectedTaskId, setTempSelectedTaskId] = useState<string | null>(null);

// アクティブなメソッドID（タブのactiveKey）
const [activeMethodId, setActiveMethodId] = useState<string | null>(null);
```

4.2 フィルター状態

```
// 検索テキスト（全カラム対象）
const [searchText, setSearchText] = useState<string>('');

// タスク名フィルター
```

```
const [selectedTaskName, setSelectedTaskName] = useState<string | undefined>(undefined);

// ステータスフィルター
const [selectedStatus, setSelectedStatus] = useState<TaskStatus | undefined>(undefined);
```

4.3 状態のリセットタイミング

モーダルが開かれた時（`open === true`）に以下をリセット：

- `tempSelectedMethodId` ← `selectedMethodId`
- `tempSelectedTaskId` ← `selectedTaskId`
- `searchText` ← `''`
- `selectedTaskName` ← `undefined`
- `selectedStatus` ← `undefined`
- `activeMethodId` ← `null`（その後、最初のメソッドを自動選択）

---

5. データ構造

5.1 データ型定義

型定義は `@/types/experimentWorker` で定義されており、コンポーネント内でインポートして使用します。

```
import { Task, TaskStatus, Method } from '@/types/experimentWorker';
```

各型の定義（参考）：

```
// @/types/experimentWorker より
interface Method {
  methodId: string;
  methodName: string;
  tasks: Task[];
}

interface Task {
  taskId: string;
```

```
  taskName: string;
  status: TaskStatus;
  progressRate: number;
  // 以下は allTasks 生成時に動的に付与
  projectId?: string;
  methodId?: string;
  methodName?: string;
}

type TaskStatus =
  | 'not_started'
  | 'started'
  | 'in_progress'
  | 'completed'
  | 'suspended'
  | 'forced_terminated';
```

5.2 データソース

メソッド一覧はpropsとして親コンポーネントから渡されます。コンポーネント内部でAPIを直接呼び出す実装はありません。

```
// メソッド一覧の取得（propsから）
const methods = useMemo<Method[]>(() => {
  return propsMethods || [];
}, [propsMethods]);

// 選択されたメソッドに紐づくタスクを取得
const allTasks = useMemo(() => {
  if (!activeMethodId || !methods) return [];

  const method = methods.find((m: Method) => m.methodId === activeMethodId);
  if (!method || !method.tasks) return [];

  // プロジェクトIDとメソッドIDを各タスクに付与
  return method.tasks.map((task: Task) => ({
    ...task,
    projectId: selectedProject,
    methodId: method.methodId,
    methodName: method.methodName,
  }));
}, [methods, selectedProject, activeMethodId]);
```

データ取得の流れ:

1. 親コンポーネントがAPIからメソッド一覧を取得し、`methods` propsとして渡す

2. `activeMethodId`（アクティブなタブのメソッドID）で特定のメソッドのタスク一覧を取得
3. タスクにプロジェクトID・メソッドID・メソッド名を付与

5.3 ステータス定義

ステータス値	表示テキスト	タグカラー
not_started	開始前	default
started	開始	blue
in_progress	作業中	orange
completed	完了	green
suspended	中断	default
forced_terminated	強制終了	red

```
const getStatusConfig = (status: TaskStatus) => {
  const configs = {
    not_started: { color: 'default', label: '開始前' },
    started: { color: 'blue', label: '開始' },
    in_progress: { color: 'orange', label: '作業中' },
    completed: { color: 'green', label: '完了' },
    forced_terminated: { color: 'red', label: '強制終了' },
    suspended: { color: 'default', label: '中断' }
  };
  return configs[status] || { color: 'default', label: '不明' };
};
```

5.4 ステータス選択肢（statusOptions）

ステータスフィルターの `Select` コンポーネントに渡す選択肢の定数です。

```
const statusOptions = [
  { label: '開始前', value: 'not_started' },
  { label: '開始', value: 'started' },
  { label: '作業中', value: 'in_progress' },
  { label: '完了', value: 'completed' },
  { label: '強制終了', value: 'forced_terminated' },
  { label: '中断', value: 'suspended' }
];
```

特徴:

- `getStatusConfig` と同じステータス一覧を網羅
- `Select` コンポーネントの `options` プロパティに直接渡す

---

6. メソッドタブ

6.1 タブ構成

```
<Tabs
  activeKey={activeMethodId || undefined}
  onChange={setActiveMethodId}
  items={methods.map((method) => ({
    key: method.methodId,
    label: method.methodName,
    children: (
      // タブ内容（検索UI + テーブル）
    ),
  }))}
  size="middle"
  type="card"
/>
```

特徴:

- `type="card"` でカードスタイル
- `size="middle"` で中サイズ
- タブ切り替えで `activeMethodId` を更新

6.2 初期化

```
// 初期化時に最初のメソッドを選択
useEffect(() => {
  if (open && methods.length > 0 && !activeMethodId) {
    setActiveMethodId(methods[0].methodId);
```

```
  }
}, [open, methods, activeMethodId]);
```

動作:

- モーダルが開かれた時、メソッドが存在する場合、最初のメソッドを自動選択
- `activeMethodId` が `null` の場合のみ実行

---

7. 検索・フィルタリング

7.1 検索UI

```
<Row gutter={[16, 16]} style={{ marginBottom: 16, marginTop: 16 }}>
  <Col xs={24} sm={8}>
    <Input
      placeholder="検索..."
      prefix={<SearchOutlined />}
      value={searchText}
      onChange={(e) => setSearchText(e.target.value)}
      allowClear
    />
  </Col>
  <Col xs={24} sm={8}>
    <Select
      placeholder="タスク名"
      style={{ width: '100%' }}
      value={selectedTaskName}
      onChange={(value) => setSelectedTaskName(value)}
      allowClear
      options={taskNameOptions}
    />
  </Col>
  <Col xs={24} sm={8}>
    <Select
      placeholder="ステータス"
      style={{ width: '100%' }}
      value={selectedStatus}
      onChange={(value) => setSelectedStatus(value as TaskStatus)}
      allowClear
      options={statusOptions}
    />
```

```
  </Col>
</Row>
```

レイアウト:

- レスポンシブグリッド（xs: 24, sm: 8）
- モバイルでは縦積み、タブレット以上で3カラム
- 各要素間のgutter: 16px

7.2 フィルタリングロジック

```
const filteredTasks = useMemo(() => {
  return allTasks.filter(task => {
    // あいまい検索（全カラム対象）
    const matchesSearch = searchText === '' ||
      task.taskName.toLowerCase().includes(searchText.toLowerCase()) ||
      getStatusConfig(task.status).label.includes(searchText) ||
      task.progressRate.toString().includes(searchText);

    // タスク名フィルタ
    const matchesTaskName = !selectedTaskName || task.taskName === selectedTaskName;

    // ステータスフィルタ
    const matchesStatus = !selectedStatus || task.status === selectedStatus;

    return matchesSearch && matchesTaskName && matchesStatus;
  });
}, [allTasks, searchText, selectedTaskName, selectedStatus]);
```

フィルター条件:

1. **全カラム検索**（`searchText`）
   - タスク名（部分一致、大文字小文字無視）
   - ステータスラベル（部分一致）
   - 進捗率（完全一致）
2. **タスク名フィルター**（`selectedTaskName`）
   - 完全一致
3. **ステータスフィルター**（`selectedStatus`）

- 完全一致

論理演算:

- 全ての条件をAND結合
- 各フィルターが未設定の場合は無視

7.3 タスク名選択肢の生成

```
const taskNameOptions = useMemo(() => {
  const uniqueNames = Array.from(new Set(allTasks.map(task => task.taskName)));
  return uniqueNames.map(name => ({ label: name, value: name }));
}, [allTasks]);
```

動作:

- アクティブなメソッドのタスクから一意なタスク名を抽出
- 重複を除去して選択肢を生成

---

8. テーブル

8.1 列定義

```
const columns: ColumnsType<Task> = [
  {
    title: '選択',
    key: 'radio',
    width: 60,
    align: 'center',
    render: (_, record: Task) => (
      <Radio
        checked={record.taskId === tempSelectedTaskId}
        onClick={(e) => {
          e.stopPropagation();
          handleRadioClick(record);
        }}
      />
    )
```

```
  },
  {
    title: 'タスクの名称',
    dataIndex: 'taskName',
    key: 'taskName',
    width: 250,
    ellipsis: true,
    sorter: (a, b) => a.taskName.localeCompare(b.taskName),
    render: (text: string) => <Text style={{ fontSize: '14px' }}>{text}</Text>
  },
  {
    title: 'ステータス',
    dataIndex: 'status',
    key: 'status',
    width: 100,
    align: 'center',
    sorter: (a, b) => a.status.localeCompare(b.status),
    render: (status: TaskStatus) => {
      const config = getStatusConfig(status);
      return (
        <Tag
          color={config.color}
          style={{ margin: 0 }}
          aria-label={`ステータス: ${config.label}`}
        >
          {config.label}
        </Tag>
      );
    }
  },
  {
    title: '進捗率',
    dataIndex: 'progressRate',
    key: 'progressRate',
    width: 100,
    align: 'center',
    sorter: (a, b) => a.progressRate - b.progressRate,
    render: (rate: number) => (
      <Text strong style={{ fontSize: '14px', color: '#212121' }}>{rate}%</Text>
    )
  }
];
```

列幅:

- 選択（ラジオボタン）: 60px
- タスク名: 250px（ellipsis対応）
- ステータス: 100px
- 進捗率: 100px

ソート:

- タスク名: localeCompare（日本語対応）
- ステータス: localeCompare（文字列順）
- 進捗率: 数値比較

配置:

- 選択: center
- タスク名: left（デフォルト）
- ステータス: center
- 進捗率: center

8.2 行のスタイリング

```
<Table
  rowClassName={(record) => {
    if (record.taskId === tempSelectedTaskId) {
      return 'selected-row';
    }
    return '';
  }}
  // ...
/>

<style jsx>{`
  :global(.selected-row) {
    background-color: #e6f7ff !important;
  }
  :global(.selected-row:hover) {
    background-color: #bae7ff !important;
  }
`}</style>
```

スタイル:

- 選択行: 背景色 `#e6f7ff`（水色）
- 選択行ホバー: 背景色 `#bae7ff`（濃い水色）

8.3 ページネーション

```
pagination={{
  pageSize: 10,
  showSizeChanger: true,
  showQuickJumper: false,
  showTotal: (total) => `全${total}件`,
  pageSizeOptions: ['10', '20', '50'],
  position: ['bottomRight'],
  locale: {
    items_per_page: '件/ページ',
    jump_to: 'ページ',
    page: ''
  }
}}
```

設定:

- デフォルトページサイズ: 10件
- ページサイズ変更: 可能（10, 20, 50）
- クイックジャンパー: なし
- 合計件数表示: あり
- 位置: 右下

---

9. イベント

9.1 イベント一覧

イベント名	トリガー	処理内容
handleRadioClick	ラジオボタンクリック	メソッドID・タスクIDを一時変数に設定
handleClearSelection	選択解除ボタンクリック	一時選択状態をクリア
handleOk	決定ボタンクリック	親に選択状態を通知してダイアログを閉じる
handleCancel	キャンセルボタンクリック	一時選択状態をリセットしてダイアログを閉じる
setActiveMethodId	タブ切り替え	アクティブメソッドIDを更新

9.2 イベント詳細

handleRadioClick（ラジオボタンクリック）

```
const handleRadioClick = (record: Task) => {
  setTempSelectedMethodId(record.methodId || null);
  setTempSelectedTaskId(record.taskId);
};
```

特徴:

- タスクから `methodId` を取得して設定
- タスクIDを設定
- 決定ボタン押下で確定

handleClearSelection（選択解除）

```
const handleClearSelection = () => {
  setTempSelectedMethodId(null);
  setTempSelectedTaskId(null);
};
```

ボタン状態:

- `disabled={!tempSelectedTaskId}` - タスク未選択時は無効

handleOk（決定）

```
const handleOk = () => {
  if (tempSelectedMethodId && tempSelectedTaskId) {
    // 選択されている場合
    onSelect(tempSelectedMethodId, tempSelectedTaskId);
  } else {
    // 未選択の場合は、空文字列を渡して親コンポーネントの選択状態をクリア
    onSelect('', '');
  }
  onClose();
};
```

動作:

- 選択されている場合: メソッドID・タスクIDを親に通知

- 未選択の場合: 空文字列を渡して親の選択状態をクリア
- ダイアログを閉じる

handleCancel（キャンセル）

```
const handleCancel = () => {
  setTempSelectedMethodId(selectedMethodId);
  setTempSelectedTaskId(selectedTaskId);
  onClose();
};
```

動作:

- 一時選択状態を元の選択状態に戻す
- ダイアログを閉じる

---

10. スタイリング

10.1 モーダルスタイル

```
<Modal
  open={open}
  onCancel={handleCancel}
  width={900}
  title={
    <Space>
      <CheckOutlined style={{ color: token.colorPrimary }} />
      <Text strong>メソッド・タスク選択</Text>
    </Space>
  }
  footer={
    <Space style={{ width: '100%', justifyContent: 'space-between' }}>
      <Button danger icon={<CloseOutlined />} onClick={handleClearSelection} disabled=
{!tempSelectedTaskId}>
        選択解除
      </Button>
      <Space>
        <Button onClick={handleCancel}>キャンセル</Button>
        <Button type="primary" onClick={handleOk} icon={<CheckOutlined />}>決定</Button>
      </Space>
```

```
    </Space>
  }
  styles={{
    body: {
      maxHeight: '70vh',
      overflowY: 'auto'
    }
  }}
>
```

特徴:

- モーダル幅: 900px
- ボディ最大高さ: 70vh（スクロール可能）
- カスタムフッター（選択解除ボタンを左側に配置）

10.2 タブスタイル

```
<Tabs
  size="middle"
  type="card"
  style={{ marginBottom: 0 }}
  tabBarStyle={{ marginBottom: 0 }}
/>
```

特徴:

- カードタイプ
- 中サイズ
- マージンをゼロに設定（モーダル内で使用するため）

10.3 テーブルスタイル

```
<Table
  size="middle"
  bordered
  scroll={{ x: 600 }}
  style={{
    background: '#ffffff',
    borderRadius: '8px'
```

```
  }}
/>
```

特徴:

- 中サイズ
- ボーダーあり
- 横スクロール対応（最小幅600px）
- 白背景、角丸8px

---

11. アクセシビリティ

11.1 キーボード操作

- Tabキーでフォーカス移動
- Enterキーで決定ボタン実行
- Escapeキーでキャンセル
- 矢印キーでテーブル行間の移動
- ラジオボタンはSpaceキーで選択

11.2 ARIA属性

```
<Radio aria-label={`タスク ${record.taskName} を選択`} />
<Tag aria-label={`ステータス: ${config.label}`} />
<Text aria-label={`進捗率 ${rate}%`} />
<Input aria-label="全カラム検索" />
<Select aria-label="タスク名で絞り込み" />
<Select aria-label="ステータスで絞り込み" />
<Table aria-label="タスク一覧" />
```

---

12. パフォーマンス考慮事項

12.1 useMemoによる最適化

```
// メソッド一覧の取得（propsから）
const methods = useMemo<Method[]>(() => {
  return propsMethods || [];
}, [propsMethods]);

// アクティブメソッドのタスク一覧の取得
const allTasks = useMemo(() => {
  // タスクにプロジェクトID・メソッドID・メソッド名を付与
}, [methods, selectedProject, activeMethodId]);

// フィルタリングされたタスクデータ
const filteredTasks = useMemo(() => {
  // フィルタリング処理
}, [allTasks, searchText, selectedTaskName, selectedStatus]);

// タスク名選択肢の生成
const taskNameOptions = useMemo(() => {
  // 一意なタスク名を抽出
}, [allTasks]);
```

最適化されている処理:

- メソッド一覧の取得（propsから）
- タスク一覧の取得（プロジェクトID・メソッドID付与）
- フィルタリング処理
- タスク名選択肢の生成

12.2 イベントハンドラーの最適化

```
onClick={(e) => {
  e.stopPropagation();  // イベント伝播を停止
  handleRadioClick(record);
}}
```

イベント伝播の制御:

- `stopPropagation()` でテーブル行のクリックイベントとの競合を防止

---

13. 備考

13.1 特記事項

1. **メソッドとタスクの同時選択**
   - ラジオボタンクリックで `methodId` と `taskId` を同時に設定
   - 決定ボタン押下で両方を親に通知

2. **タブ切り替えと選択状態**
   - タブを切り替えても一時選択状態は維持される
   - ただし、表示されるタスク一覧は切り替わる

3. **選択解除機能**
   - 選択解除ボタンで一時選択状態をクリア
   - 決定ボタン押下で親の選択状態もクリア（空文字列を渡す）

4. **初期化動作**
   - モーダル表示時、最初のメソッドを自動選択
   - フィルター状態をリセット
   - 一時選択状態を現在の選択状態で初期化

5. **ProgressCheckTabとの構造の一致**
   - メソッドタブ + タスクテーブルの構造
   - 検索・フィルタリングのUI
   - テーブルの列定義とスタイリング
   - 再利用性を考慮した設計

6. **ProjectSelectionModalとの違い**
   - タブによる複数メソッドの切り替え
   - メソッドとタスクを同時に選択
   - より複雑な選択ワークフロー

7. **モーダル幅の選択理由**

- 900px: テーブルの4列を快適に表示できる幅
- 検索UIの3カラムレイアウトに適している

8. **スタイルの埋め込み**
   - `<style jsx>` で選択行のスタイルを定義
   - `:global()` でグローバルスコープに適用
   - `!important` で優先度を確保

13.2 他のコンポーネントとの関係

コンポーネント	関係	説明
ExperimentRecordTab	利用元	メソッド・タスク選択時にこのモーダルを開く
ProgressCheckTab	利用元	メソッド・タスク選択時にこのモーダルを開く
InstructionConfirmationTab	利用元	メソッド・タスク選択時にこのモーダルを開く
ProjectSelectionModal	兄弟コンポーネント	プロジェクト・メソッド選択用の別モーダル

利用元コンポーネントの責務:

- APIからメソッド一覧（`Method[]`）を取得し、`methods` propsとして渡す
- 選択確定後の `onSelect(methodId, taskId)` コールバックを実装する
- モーダルの開閉状態（`open`）を管理する

---

14. 最終更新

- **最終更新日**: 2026-03-11
- **更新内容**: ソースコードとの差分を修正（`methods` props追加、型定義インポート明記、データソース説明修正、`statusOptions` 追加、`Tag` の `style` 追加、`useMemo` 依存配列修正）